Decision Tree classifier

In this notebook, we will use the scikit-learn library to perform a decision tree based classification of medical data. The dataset used in this notebook is the public [Wisconsin Diagnostic Breast Cancer (WDBC) dataset (https://archive.ics.uci.edu/ml/datasets/breast+cancer+wisconsin+(diagnostic))](https://archive.ics.uci.edu/ml/datasets/breast+cancer+wisconsin+(diagnostic)), which contains 569 examples of malignant and benign tumor cells.

Step 1: Loading Data

Load the WDBC data file into a Pandas DataFrame, a 2-dimensional labeled data structure with columns of potentially different types.

In [1]:
```
import pandas as pd

df = pd.read_csv('https://archive.ics.uci.edu/ml/machine-learning-databases/breast-can
cer-wisconsin/wdbc.data', header=None)
df.columns = ['id', 'diagnosis', 'radius_mean', 'texture_mean', 'perimeter_mean',
       'area_mean', 'smoothness_mean', 'compactness_mean', 'concavity_mean',
       'concave points_mean', 'symmetry_mean', 'fractal_dimension_mean',
       'radius_se', 'texture_se', 'perimeter_se', 'area_se', 'smoothness_se',
       'compactness_se', 'concavity_se', 'concave points_se', 'symmetry_se',
       'fractal_dimension_se', 'radius_worst', 'texture_worst',
       'perimeter_worst', 'area_worst', 'smoothness_worst',
       'compactness_worst', 'concavity_worst', 'concave points_worst',
       'symmetry_worst', 'fractal_dimension_worst']

# print(df.shape)
print('Number of instances = %d' % (df.shape[0]))
print('Number of attributes = %d' % (df.shape[1]))

# df.head()
```

```
Number of instances = 569
Number of attributes = 32
```

Step 2: Review the data

In [2]:
```
# print a statistics summary for all the numeric features of the dataframe.

df.describe()
```

Out[2]:

	id	radius_mean	texture_mean	perimeter_mean	area_mean	smoothness_mean	compactness_m
count	5.690000e+02	569.000000	569.000000	569.000000	569.000000	569.000000	569.000
mean	3.037183e+07	14.127292	19.289649	91.969033	654.889104	0.096360	0.104
std	1.250206e+08	3.524049	4.301036	24.298981	351.914129	0.014064	0.052
min	8.670000e+03	6.981000	9.710000	43.790000	143.500000	0.052630	0.019
25%	8.692180e+05	11.700000	16.170000	75.170000	420.300000	0.086370	0.064
50%	9.060240e+05	13.370000	18.840000	86.240000	551.100000	0.095870	0.092
75%	8.813129e+06	15.780000	21.800000	104.100000	782.700000	0.105300	0.130
max	9.113205e+08	28.110000	39.280000	188.500000	2501.000000	0.163400	0.345

8 rows × 31 columns

In [3]:
```
df.head()
# df.tail(10)
```

Out[3]:

	id	diagnosis	radius_mean	texture_mean	perimeter_mean	area_mean	smoothness_mean	compactness_
0	842302	M	17.99	10.38	122.80	1001.0	0.11840	0.2
1	842517	M	20.57	17.77	132.90	1326.0	0.08474	0.0
2	84300903	M	19.69	21.25	130.00	1203.0	0.10960	0.1
3	84348301	M	11.42	20.38	77.58	386.1	0.14250	0.2
4	84358402	M	20.29	14.34	135.10	1297.0	0.10030	0.1

5 rows × 32 columns

In [4]:
```
df.groupby('diagnosis').size()
```

Out[4]:
```
diagnosis
B    357
M    212
dtype: int64
```

In [5]:
```
df.isnull().sum()
```

Out[5]:
```
id                         0
diagnosis                  0
radius_mean                0
texture_mean               0
perimeter_mean             0
area_mean                  0
smoothness_mean            0
compactness_mean           0
concavity_mean             0
concave points_mean        0
symmetry_mean              0
fractal_dimension_mean     0
radius_se                  0
texture_se                 0
perimeter_se               0
area_se                    0
smoothness_se              0
compactness_se             0
concavity_se               0
concave points_se          0
symmetry_se                0
fractal_dimension_se       0
radius_worst               0
texture_worst              0
perimeter_worst            0
area_worst                 0
smoothness_worst           0
compactness_worst          0
concavity_worst            0
concave points_worst       0
symmetry_worst             0
fractal_dimension_worst    0
dtype: int64
```

Step 3: Choosing features

It's important to remove redundant and irrelevant attributes from the dataset to simplify classification (reduce overfitting, reduce curse of dimensionality)

- Overfitting: An over-complex model fits very accurately the observed data (very small training error), but is not able to provide accurate predictions for unseen data
- Curse of dimensionality: Volumes in high-dimensional spaces grow very fast, and data becomes very sparse

Select the columns that are used to make predictions (features), by convention this data is called X.

Select multiple features by providing a list of column names inside brackets. Each item in that list should be a string (with quotes).

In [6]:
```
# print the list of columns in the dataset
# df.columns

# create a list with the names of the columns you want in X
# features = ['radius_mean', 'texture_mean', 'perimeter_mean', 'area_mean',
#             'smoothness_mean', 'compactness_mean', 'concavity_mean',
#             'concave points_mean', 'symmetry_mean', 'fractal_dimension_mean',
#             'radius_se', 'texture_se', 'perimeter_se', 'area_se', 'smoothness_se',
#             'compactness_se', 'concavity_se', 'concave points_se', 'symmetry_se',
#             'fractal_dimension_se', 'radius_worst', 'texture_worst',
#             'perimeter_worst', 'area_worst', 'smoothness_worst',
#             'compactness_worst', 'concavity_worst', 'concave points_worst',
#             'symmetry_worst', 'fractal_dimension_worst']
# X = df[features]

# X = df.drop("diagnosis",axis = 1)

X = df.drop(["id", "diagnosis"],axis = 1)
print(X.shape, type(X))
```

```
(569, 30) <class 'pandas.core.frame.DataFrame'>
```

In [7]:
```
X.head()
```

Out[7]:

	radius_mean	texture_mean	perimeter_mean	area_mean	smoothness_mean	compactness_mean	concavity_mear
0	17.99	10.38	122.80	1001.0	0.11840	0.27760	0.3001
1	20.57	17.77	132.90	1326.0	0.08474	0.07864	0.0869
2	19.69	21.25	130.00	1203.0	0.10960	0.15990	0.1974
3	11.42	20.38	77.58	386.1	0.14250	0.28390	0.2414
4	20.29	14.34	135.10	1297.0	0.10030	0.13280	0.1980

5 rows × 30 columns

Step 4: Selecting the prediction target

Select the column you want to predict, which is called the prediction target. By convention, the prediction target is called y.

This single column is stored in a Series, which is broadly like a DataFrame with only a single column of data.

In [8]:
```
# Selecting with the dot notation
# Y = df.diagnosis

# Selecting with a column list
Y = df['diagnosis']
print(Y.shape, type(Y))
```

```
(569,) <class 'pandas.core.series.Series'>
```

Step 5: Create the train / test split

In order to test the model with regard to its predictive accuracy, split data into a training portion and a test portion, so that the model can be trained on one dataset, but tested on data it hasn't seen before.

In [9]:
```
from sklearn.model_selection import train_test_split

# split data into training and test data, for both features and target.
# The split is based on a random number generator. Supplying a numeric value to
# the random_state argument to get the same split every time we run this script.

x_train,x_test,y_train,y_test = train_test_split(X,Y,test_size = 0.2,random_state = 4
2)
```

Step 6: Specify and fit the model

In [10]:
```
from sklearn.tree import DecisionTreeClassifier
from sklearn import tree

# Define model
model = tree.DecisionTreeClassifier(max_depth= 3 , min_samples_leaf=12)

# Fit model
model.fit(x_train,y_train)
```

Out[10]:
```
DecisionTreeClassifier(max_depth=3, min_samples_leaf=12)
```

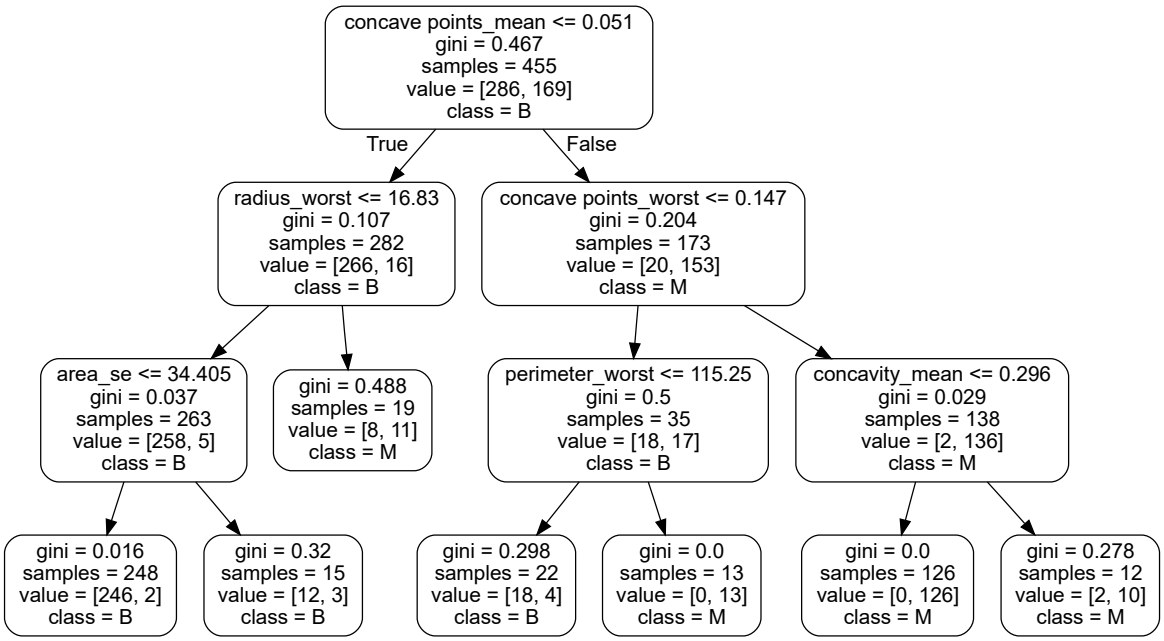
In [11]:
```
import graphviz

attributes_name = X.columns.values

def Decision_Tree_draw (mdl,names):
    ds = tree.export_graphviz(mdl,out_file = None,
                              feature_names = names,
                              class_names = mdl.classes_,
                              filled = False, rounded = True,
                              special_characters =False)
    grp = graphviz.Source(ds)
    return grp

Decision_Tree_draw(model,attributes_name)
```

Out[11]:



Step 7: Make predictions using the model

Make predictions with the model's `predict` command, using `x_test` as the data. Save the results to a variable called `y_pred`.

In [12]:
```
y_pred = model.predict(x_test)
```

Step 8: Evaluate the predictive accuracy of the classifier

A confusion matrix can be used to evaluate the predictive accuracy of the classifier. By definition a confusion matrix C is such that $C_{i,j}$ is equal to the number of observations known to be in group i and predicted to be in group j. All the diagonal elements denote correctly classified outcomes. The misclassified outcomes are represented on the off diagonals of the confusion matrix.

Confusion matrix for a binary classifier:

		PREDICTED CLASS	
		Class=Yes	Class=No
ACTUAL CLASS	Class=Yes	a (TP)	b (FN)
	Class=No	c (FP)	d (TN)

From: Tan, Steinbach, Karpatne, Kumar, Introduction to Data Mining 2nd Edition, Pearson 2019

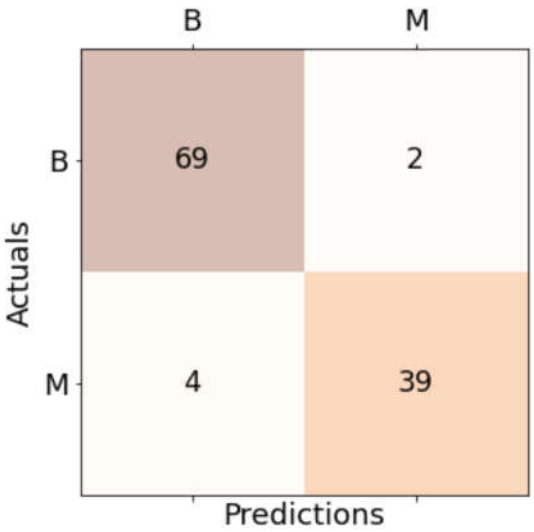
In [13]:
```
from sklearn import metrics
from sklearn.metrics import confusion_matrix
import matplotlib.pyplot as plt
#
# Calculate the confusion matrix
#
conf_matrix = confusion_matrix(y_true=y_test, y_pred=y_pred)

# List of labels to index the matrix. This may be used to reorder or select a subset o
f labels.
# If None is given, those that appear at least once in y_true or y_pred are used in so
rted order.
# conf_matrix = confusion_matrix(y_test, y_pred, labels=['M', 'B'])

# print(conf_matrix)

#
# Print the confusion matrix using Matplotlib
#
fig, ax = plt.subplots(figsize=(5, 5))
ax.matshow(conf_matrix, cmap=plt.cm.Oranges, alpha=0.3)
for i in range(conf_matrix.shape[0]):
    for j in range(conf_matrix.shape[1]):
        ax.text(x=j, y=i,s=conf_matrix[i, j], va='center', ha='center', size='xx-large
')

plt.xlabel('Predictions', fontsize=18)
plt.ylabel('Actuals', fontsize=18)
# plt.title('Confusion Matrix', fontsize=18)
ax.set_xticks(range(2))
ax.set_yticks(range(2))
ax.set_xticklabels(['B', 'M'], fontsize=18)
ax.set_yticklabels(['B', 'M'], fontsize=18)
plt.show()
```



Classification Evaluation metrics

$$Accuracy = \frac{TP+TN}{TP+TN+FP+FN}$$

Accuracy represents the number of correctly classified data instances over the total number of data instances. Accuracy is not appropriate for unbalanced class label distribution and when misclassification of objects of a given class is more important (e.g. malignant tumor classified as benign).

$$Precision = \frac{TP}{TP+FP}$$

Precision is a valid choice of evaluation metric when we want to be very sure of our prediction.

$$Recall = \frac{TP}{TP+FN}$$

Recall is a valid choice of evaluation metric when we want to capture as many positives as possible.

In [14]:
```
from sklearn import metrics
from sklearn.metrics import precision_score, recall_score, accuracy_score
from sklearn.metrics import precision_recall_fscore_support as score

# print(precision_score(y_test, y_pred, average=None))
# precision, recall, fscore, support = score(y_test, y_pred)

print(metrics.classification_report(y_test, y_pred, digits=4))
```

```
              precision    recall  f1-score   support

           B     0.9452    0.9718    0.9583        71
           M     0.9512    0.9070    0.9286        43

    accuracy                         0.9474       114
   macro avg     0.9482    0.9394    0.9435       114
weighted avg     0.9475    0.9474    0.9471       114
```

Step 9: Tuning the classifier

The vast majority of predictive models have parameters. A parameter is a number to pick in advance that determines some aspect of how the model behaves. For example, in the Decision Tree classification algorithm, *max_depth* is a parameter that we have to pick that determines how deep the tree can be. By picking different values of *max_depth*, we create different classifiers that make different predictions.

To tune the model, we want to maximize the performance of our classifier on data it hasn't seen yet. But **we cannot use our test data set in the process of building our model** (tuning is a part of model training), so we split the training data into two subsets, use one to train the model, and then use the other to evaluate it. The subset of training data used for evaluation is often called the **validation set**.

In a **k-fold cross-validation** strategy, the overall training data is randomly partitioned into k equal sized chunks (typically 10). Of the k chunks, one is retained as the validation data for testing the model, and the remaining k − 1 chunks are used as training data. The cross-validation process is then repeated k times, with each of the k chunks used exactly once as the validation data. The k results can then be averaged to produce a single estimation of the accuracy of the classifier for a given value of the parameter.

We can use cross-validation to calculate an accuracy for each value of the parameter in a reasonable range, and then pick the value of the parameter that gives us the best accuracy.

In [15]:
```
%%shell
jupyter nbconvert --to html /DecisionTreeClassifier.ipynb
```

```
[NbConvertApp] Converting notebook /DecisionTreeClassifier.ipynb to html
[NbConvertApp] Writing 356439 bytes to /DecisionTreeClassifier.html
```

Out[15]:

In [15]: